ESCUELA DE INGENIERÍA ELÉCTRICA

PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO

Guía 8

FastApi

Curso: EIE390 - Robótica e Inteligencia Artificial
Profesor: Guillermo Cid Ampuero

Objetivos

- **Objetivo 1:** Comprender la arquitectura cliente-servidor mediante la implementación de una API utilizando el framework FastAPI en Python.
- **Objetivo 2:** Establecer comunicación bidireccional por puerto Serial para controlar actuadores físicos (LEDs) desde una interfaz web gráfica (Frontend) mediante peticiones HTTP.
- **Objetivo 3:** Desarrollar un sistema de monitoreo en tiempo real, procesando lecturas analógicas (ADC) en la Raspberry Pi Pico y visualizándolas en el navegador utilizando técnicas de *Polling* con JavaScript.

Repositorio del Curso

1 Integración de Hardware y Software: Introducción a FastAPI

1.1 ¿Qué es una API en el contexto de la Robótica y el IoT?

En proyectos de ingeniería, no basta con que un microcontrolador adquiera datos del entorno; esta información debe ser visualizada, procesada o almacenada por otros sistemas. Una API (Interfaz de Programación de Aplicaciones) actúa como un puente estandarizado que permite a diferentes aplicaciones de software comunicarse entre sí.

Para este laboratorio, utilizaremos **FastAPI**, un framework de Python moderno, rápido y altamente eficiente para la construcción de APIs. Su principal ventaja en la academia y la industria radica en su simplicidad para levantar un servidor web robusto en pocas líneas de código, permitiendo exponer los datos de nuestros sensores físicos hacia aplicaciones web o de escritorio.

1.2 Arquitectura del Sistema de Adquisición de Datos

Para lograr que el dato físico del sensor llegue al navegador web del computador, implementaremos una arquitectura de tres etapas:

1. **Adquisición (Capa Física):** La Raspberry Pi Pico lee la variación de voltaje del sensor a través de su pin ADC (Analógico a Digital) o procesa entradas digitales, enviando o recibiendo la información hacia el computador utilizando comunicación Serial a través del cable USB.

2. **Puente de Comunicación (Middleware):** Un script de Python ejecutándose en el computador utiliza la librería `pyserial` para 'escuchar' y 'escribir' en el puerto COM/-Serial, capturando el flujo de datos que envía la placa o enviando comandos hacia ella en tiempo real.

3. **Servidor Web (Backend):** En ese mismo entorno de Python, FastAPI gestiona los datos físicos y los publica en rutas web específicas (*endpoints*). Así, al interactuar con una dirección desde el navegador, el servidor responderá con el valor actual del sensor o ejecutará una acción física.

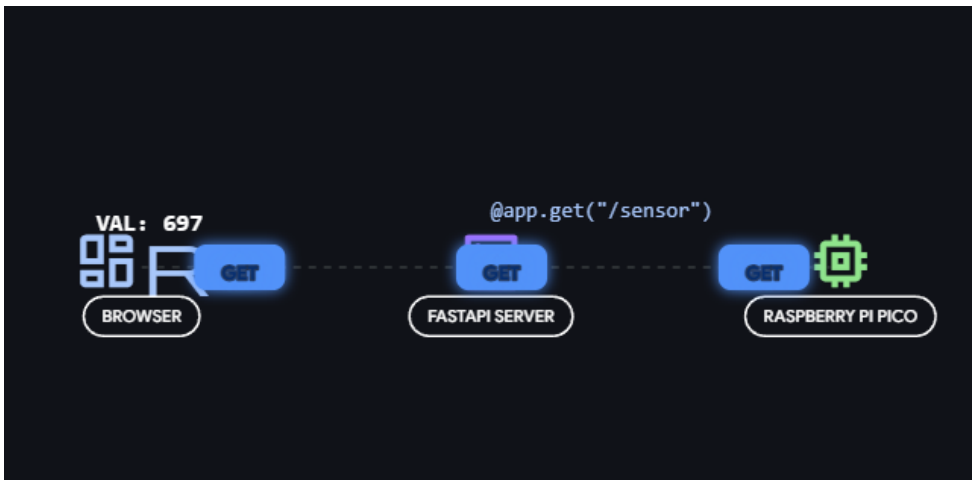

Figura 1: Arquitectura del sistema: Flujo de datos bidireccional entre la Raspberry Pi Pico y el cliente web mediante FastAPI.

1.3 Métodos de Petición HTTP: GET y POST

Para que el Frontend (la página web) y el Backend (nuestro servidor FastAPI) se entiendan, utilizan el protocolo HTTP. Este protocolo define diferentes 'métodos' o verbos que indican la intención de la comunicación. En el contexto de control por hardware, nos centraremos en los dos más importantes:

- **Método GET (Obtener):** Se utiliza exclusivamente para **solicitar o consultar datos** del servidor sin modificar su estado. En nuestro entorno de robótica, utilizamos GET para la lectura de instrumentación. Por ejemplo, cuando el navegador hace una petición GET a la ruta `/sensor`, el servidor lee el estado actual del potenciómetro a través del puerto Serial y devuelve ese valor. No se altera el entorno físico, solo se observa.
- **Método POST (Enviar):** Se utiliza para **enviar datos** al servidor con el objetivo de crear un registro o **modificar un estado**. En el control de hardware, POST es el método semánticamente correcto para encender motores, activar relés o cambiar el estado de un LED. A través de POST, el cliente (navegador) le envía una orden explícita a la API (por ejemplo, el dato de encendido), y la API traduce esa orden escribiendo un comando en el puerto Serial hacia el microcontrolador.

Nota Práctica

En etapas iniciales y pruebas rápidas desde la barra de direcciones del navegador, es común utilizar el método **GET** incluso para enviar comandos simples (ej. `/led/1`). Sin embargo, a medida que los sistemas de control se vuelven más complejos y seguros, la convención de ingeniería de software exige separar estrictamente las lecturas (GET) de las acciones de control físico (POST).

1.4 Consideraciones de Red en el Laboratorio

Debido a que trabajaremos en los equipos institucionales del laboratorio, es fundamental comprender cómo se enrutará nuestro servidor para evitar conflictos con el firewall o las políticas de seguridad de la red Ethernet de la universidad.

Para garantizar el correcto funcionamiento sin requerir permisos de administrador de red, el servidor de FastAPI se ejecutará exclusivamente en un entorno **Localhost**. Esto significa que operará sobre la dirección IP de *loopback* **127.0.0.1**.

Bajo esta configuración, el tráfico de red generado por la API no sale por la tarjeta de red física del computador, sino que se gestiona de manera interna en la memoria de la máquina. Por lo tanto, el estudiante deberá levantar y consultar su servidor estrictamente desde el mismo equipo físico en el que está trabajando.

2 Actividad 1: Mi primer Servidor con FastAPI

2.1 Objetivo

Instalar el entorno de trabajo y desplegar un servidor web básico para comprender el concepto de *endpoints* y respuestas en formato JSON.

2.2 Instrucciones

1. **Entorno:** Abra una terminal y asegúrese de tener instaladas las librerías necesarias: `pip install fastapi uvicorn`.
2. **Código:** Cree un archivo llamado `main.py` y pegue el código proporcionado.
3. **Ejecución:** Corra el servidor ejecutando directamente el script de Python con el siguiente comando en la terminal:
   `python main.py`
4. **Prueba:** Abra su navegador en `http://127.0.0.1:8000/`. Luego, acceda a la documentación automática en `http://127.0.0.1:8000/docs`.

Explicación FastAPI y Red

@app.get('/'): Indica que cuando alguien acceda a la raíz del sitio, se ejecute la función siguiente.
return {...}: FastAPI convierte automáticamente los diccionarios de Python a formato JSON.
--host 127.0.0.1: Parámetro de Uvicorn que asegura que el servidor solo escuche peticiones internas del mismo computador, evitando bloqueos por el firewall institucional.

3 Actividad 2: Control de Actuadores desde Interfaz Web

3.1 Objetivo

Implementar un sistema de control donde una interfaz HTML envíe comandos a la Raspberry Pi Pico a través de FastAPI para conmutar un LED físico. Comprender la relación entre los eventos de la interfaz web y los parámetros de ruta en la API para poder escalar el proyecto.

3.2 Análisis del Código: ¿Cómo se comunican?

Para poder expandir este sistema, es vital comprender cómo se enlaza el clic en la página web con el servidor Python.

1. El Botón HTML (Frontend)

Observe la siguiente línea en el archivo `index.html`:

```
<button class='btn btn-on' onclick='fetch('/led/1')'>Encender LED</button>
```

- **onclick**: Es un evento de HTML que detecta el momento exacto en que el usuario hace clic en el botón.
- **fetch('/led/1')**: Es una función de JavaScript que realiza una petición de red. En este caso, le avisa a nuestro servidor FastAPI diciendo: *'¡Se requiere acceder a la ruta /led/1!'*.

2. El Endpoint en FastAPI (Backend)

Observe cómo FastAPI recibe esa petición en `main.py`:

```
@app.get('/led/{estado}')
def controlar_led(estado: str):
    if estado == '1':
        ser.write(b'E')
```

- **{estado}**: Se conoce como *Variable de Ruta* (Path Parameter). FastAPI es inteligente: si el botón web pide `/led/1`, FastAPI captura ese `1` de la URL y lo guarda automáticamente en la variable local `estado`.
- **ser.write(b'E')**: Para enviar datos por el cable USB, la librería Serial exige que enviemos 'Bytes', no texto normal. La letra `b` antes de las comillas (`b'E'`) convierte el carácter en un byte bruto que la Pico puede procesar inmediatamente.

3.3 Instrucciones

1. **Archivos:** Asegúrese de tener `main.py` e `index.html` en la misma carpeta.
2. **Ejecución:** Levante el servidor utilizando `python main.py`.
3. **Hardware:** Conecte el LED y cargue el script de MicroPython en la Pico. Recuerde detener el script en Thonny antes de usar la web.
4. **Prueba Inicial:** Verifique en su navegador (`http://127.0.0.1:8000/`) que los botones encienden y apagan el LED correctamente. Observe la terminal de Python para ver los mensajes de depuración.

Reto Práctico: Expandiendo el Sistema

Ahora que comprende cómo fluye la información desde el clic hasta la placa física, modifique el código para controlar **un segundo LED** conectado al pin GP14 de la Pico. Para lograrlo, deberá:

1. **En HTML (`index.html`):** Crear dos botones nuevos ('Encender LED 2' y 'Apagar LED 2') que utilicen `fetch()` para apuntar a nuevas rutas (por ejemplo, `/led2/1` y `/led2/0`).
2. **En FastAPI (`main.py`):** Crear un nuevo endpoint que capture esa ruta (ej. `@app.get('/led2/{estado}')`) y envíe comandos diferentes por el puerto serial (por ejemplo, las letras `b'X'` y `b'Y'`).
3. **En MicroPython (`ejercicio2_pico.py`):** Instanciar el pin GP14, agregar más condicionales `elif` para capturar las nuevas letras y encender/apagar el segundo LED según corresponda.

4 Anexo (Opcional): Acceso a la API desde un Dispositivo Móvil

4.1 Objetivo

Convertir nuestro servidor local en un servidor accesible dentro de la misma red local (WLAN/LAN). Esto permitirá interactuar con la interfaz gráfica y controlar los actuadores de la Raspberry Pi Pico directamente desde un teléfono celular u otro equipo.

Nota sobre Redes Institucionales

Importante: Esta actividad está pensada principalmente para ser replicada en sus hogares. En los computadores del laboratorio de la universidad, es altamente probable que las políticas de seguridad (Firewall institucional o aislamiento de clientes Wi-Fi) bloqueen las conexiones entrantes desde teléfonos móviles. Sin embargo, puede intentar la configuración si el equipo lo permite.

4.2 Paso 1: Modificar el Host en FastAPI

Por defecto, nuestro servidor se ejecuta bajo la dirección IP `127.0.0.1` (Localhost), lo que significa que **solo** escucha peticiones internas creadas por el mismo computador. Para indicar que debe escuchar peticiones externas provenientes de cualquier tarjeta de red, debemos cambiar esta dirección a `0.0.0.0`.

Abra su archivo `main.py` y modifique exclusivamente la última línea del bloque de ejecución:

```
if __name__ == '__main__':
    # host='0.0.0.0' permite conexiones externas en la red local
    uvicorn.run('main:app', host='0.0.0.0', port=8000, reload=True)
```

4.3 Paso 2: Obtener la Dirección IP del Servidor

Para que su celular encuentre al servidor, necesita conocer su 'dirección' física en la red local (Dirección IPv4).

1. Abra una nueva ventana de terminal (Símbolo del sistema o PowerShell).
2. Escriba el comando `ipconfig` y presione la tecla `Enter`.
3. Busque el adaptador de red que le provee internet en ese momento (generalmente denominado 'Adaptador de LAN inalámbrica Wi-Fi' o 'Adaptador de Ethernet').
4. Anote el número de 4 bloques que aparece en la fila **Dirección IPv4** (Por ejemplo: `192.168.1.15` o `10.0.0.50`).

4.4 Paso 3: Configurar el Firewall (Solo Windows)

Al ejecutar su código `main.py` modificado por primera vez, es muy probable que el sistema operativo lance una ventana de alerta de seguridad de **Windows Defender Firewall**.

- Asegúrese de marcar la casilla para permitir conexiones en **Redes privadas**.
- Haga clic en **Permitir acceso**. Si omite este paso o cancela el cuadro de diálogo, el firewall bloqueará silenciosamente la petición de su celular y la página nunca cargará.

4.5 Paso 4: Conexión desde el Teléfono Móvil

1. Verifique que su teléfono celular esté conectado **exactamente a la misma red Wi-Fi** que su computador servidor.
2. Abra un navegador web (Chrome, Safari, Firefox, etc.) en su dispositivo móvil.
3. En la barra de direcciones (no en el buscador de Google), escriba la Dirección IPv4 obtenida en el Paso 2, seguida de dos puntos (`:`) y el puerto `8000`.
   Ejemplo: `http://192.168.1.15:8000`
4. Presione 'Ir' o 'Enter'. Debería visualizar la interfaz HTML adaptada a la pantalla de su celular y poder encender/apagar los LEDs del laboratorio al tocar los botones correspondientes.

5 Actividad 3: Monitoreo de Sensores (Lectura ADC)

5.1 Objetivo

Cerrar el ciclo de control bidireccional mediante la recepción de datos. La Raspberry Pi Pico enviará el valor de un potenciómetro a la API, la cual lo procesará y pondrá a disposición de la interfaz gráfica en tiempo real.

5.2 Instrucciones

1. **Lectura:** Configure el pin GP26 como ADC en la Pico. El script debe escalar el valor y enviarlo constantemente hacia la computadora utilizando la función `print()`.
2. **Endpoint de Recepción:** Reemplace el archivo `main.py` con el código proporcionado. Analice cómo la nueva ruta captura los datos seriales.
3. **Interfaz Web:** Reemplace el archivo `index.html`. Observe cómo se ha incorporado un script de JavaScript para actualizar los datos visualmente.

5.3 Análisis del Código: Recepción y Flujo de Datos

En esta etapa, el flujo de información se invierte: la placa actúa como transmisor continuo y el servidor web como receptor. Para que la visualización sea fluida y sin retrasos (*lag*), aplicaremos técnicas de sincronización en ambas capas del software.

1. El Endpoint de Recepción (Backend en Python)

Observe la lógica dentro de la ruta `@app.get('/sensor')` en `main.py`:

```
if ser.in_waiting > 0:
    lineas = ser.readlines()
    if lineas:
        ultima_linea = lineas[-1].decode('utf-8').strip()
        if ultima_linea.isdigit():
            ultimo_valor_sensor = ultima_linea
```

- `ser.readlines()`: Debido a que la Pico envía datos a gran velocidad, estos hacen 'fila' en un *buffer* del puerto USB. Esta función lee **todos** los datos acumulados de un solo golpe, vaciando la fila.
- `lineas[-1]`: En Python, el índice `-1` accede al último elemento de una lista. Esto garantiza que descartemos los datos viejos y guardemos únicamente la lectura más fresca y reciente del hardware.
- `isdigit()`: Valida que el texto recibido sea estrictamente numérico. Si al conectar el cable se genera ruido eléctrico (caracteres corruptos), esta línea evita que el servidor falle.

2. El Ciclo de Consulta o *Polling* (Frontend en HTML)

Observe el bloque de código JavaScript `<script>` al final del archivo `index.html`:

```
setInterval(async () => {
    const respuesta = await fetch('/sensor');
    const datos = await respuesta.json();
    if (datos.valor && datos.valor !== '') {
        document.getElementById('valor_potenciometro').innerText = datos.valor;
    }
}, 500);
```

- `setInterval(..., 500)`: Crea un bucle infinito en el navegador que ejecuta las instrucciones cada 500 milisegundos. Esta técnica de preguntar periódicamente al servidor se conoce en redes como ***Polling***.
- `fetch('/sensor')`: Realiza una petición GET automática a nuestro servidor FastAPI solicitando el último valor registrado.
- `document.getElementById(...)`: Busca en la estructura de la página web el elemento visual (la etiqueta `<span>`) y reemplaza su texto con el dato numérico, actualizando la pantalla sin necesidad de recargar la página completa.

6 Referencias

1 Link 1

2 Link 2

3 Link 3

4 Link 4